certora

Security Assessment
Final Report

Light – Relax Ata Decompress Signer Check

04.2026

Prepared for Light Protocol

certora

Table of content

certora

Project Summary

Project Scope

Project Name	Repository (link)	Latest Commit Hash	Platform
Light Protocol	https://github.com/Lightprotocol/light-protocol	a9561a8	Solana

Project Overview

This document describes the specification and verification of **Light Protocol - Relax Ata Decompress Signer Check** update using manual code review findings. The work was undertaken from **08.04.2026** to **18.04.2026**

The changes made in the following commit are included in the review scope.

The team performed a manual audit of all the **Rust** programs**.** During the verification process and the manual audit, the Certora team discovered bugs in the Rust program code, as listed on the following page.

Protocol Overview

The compressed token program is a program that provides similar functionality to the SPL token program, while utilizing the underlying Light System Program in order to save on rent for token accounts.
The code being audited contains an upgrade to the existing compressed token program.

The update scope consists of making an ATA decompress permissionless since

destinations are deterministic PDAs. Adding bloom filter idempotency check for single-input ATA decompress so already-spent accounts return Ok as a no-op.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

While auditing the changes, we've investigated various threats and attack vectors, among them:

Decompress to CToken with different owner (fund theft)

- Forge "is_ata=true" on a non-ATA compressed account
- Supply wrong "owner_index/bump" to redirect ATA derivation to attacker's wallet
- Pass attacker-controlled CToken as decompress destination
- Decompress same compressed account into multiple CToken accounts

Permissionless decompress harms ATA owner (griefing)

- Force-freeze the owner's CToken by decompressing an account with `is_frozen=true`
- Loss of funds via decompress timing or redirection
- Inflate delegated_amount or withheld_transfer_fee on the destination CToken
- Front-run owner's decompress by making destination CToken non-fresh

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	0	0	0
High	1	1	0
Medium	0	0	0
Low	1	1	1
Informational	1	1	1
Total	3	3	2

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

certora

Detailed Findings

ID	Title	Severity	Status
H-01	Permissionless ATA Decompress Can Increase Owner's Frozen Token Balance	High	Partially fixed
L-01	Idempotent ATA decompress accepts stale replays without re-validating parameters	Low	Fixed

High Severity Issues

H-01 Permissionless ATA Decompress Can Increase Owner's Frozen Token Balance		
Severity: **High**	Impact: **High**	Likelihood: **Medium**
Files: programs/compressed-token/program/src/shared/token_input.rs	Status: Partially fixed	

Description:

Since ATA decompress requires no signer authorization, anyone can decompress a frozen compressed account onto its corresponding CToken account. If the ATA was previously closed (e.g. by a forester's compress-and-close), the owner may reinitialize it in an unfrozen state and begin using it normally — receiving transfers and accumulating a balance. At that point, anyone can permissionlessly decompress the original frozen compressed account into this ATA, overriding its state to frozen and locking both the decompressed tokens and any funds the owner received since reinitialization.

Exploit Scenario:

1. Alice has a frozen CToken ATA holding 100 tokens
2. A forester compresses and closes Alice's frozen ATA
3. Alice reinitializes her ATA in an unfrozen state and begins using it normally
4. Alice receives 500 tokens from various transfers into her ATA
5. Bob permissionlessly decompresses Alice's original frozen compressed account (100 tokens) into her ATA
6. The decompression overrides the ATA state to frozen, locking all 600 tokens until the freeze authority acts

Recommendations: Don't allow permissionless decompression, or highlight the potential impact in the documentation

Customer's response: Partially fixed by documenting it in 076162d.

Fix Review: Fix confirmed.

Low Severity Issues

L-01 Idempotent ATA decompress accepts stale replays without re-validating parameters		
Severity: **Low**	Impact: **Low**	Likelihood: **Low**
Files: programs/compressed-token/program/src/compressed_token/transfer2/processor.rs	Status: Fixed	

Description:
In `processor.rs`, when `check_ata_decompress_idempotent()` detects that the single ATA input is already spent in the input-queue bloom filter, the function returns `Ok(true)` and triggers an early `return Ok(())`. This short-circuits all subsequent validation. The root cause is that the idempotent path does not re-check the decompress amount, destination account, or mint before succeeding. Consequently, after a valid ATA decompress completes, a stale replay with the same input but a wrong recipient or wrong amount would still succeed as a no-op rather than fail with a validation error.

Recommendations:
Add a dedicated validation function called only when `check_ata_decompress_idempotent()` returns `Ok(true)`. Before returning success, verify:

- Decompress amount matches the input amount
- Destination account is the correct ATA for the recorded wallet owner
- Destination mint matches the compression mint

If any check fails, return the normal validation error instead of succeeding as a no-op.

Customer's response:

Fixed in e378d776c18409db18052c932f22988e385a67ce.

Fix Review:

Fix confirmed.

Informational Issues

I-01. Idempotent ATA detection accepts overly broad TLV shape

Description:
`is_idempotent_ata_decompress()` function treats an input as idempotent if `inputs.in_tlv contains any flattened `CompressedOnly` entry with `is_ata = true`. The check does not ensure the flattened TLV set contains exactly one entry for the single-input path.

Recommendation:
Tighten `is_idempotent_ata_decompress()` to require exactly one flattened input TLV entry.

Customer's response:
Fixed in e343709.

Fix Review:

Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.